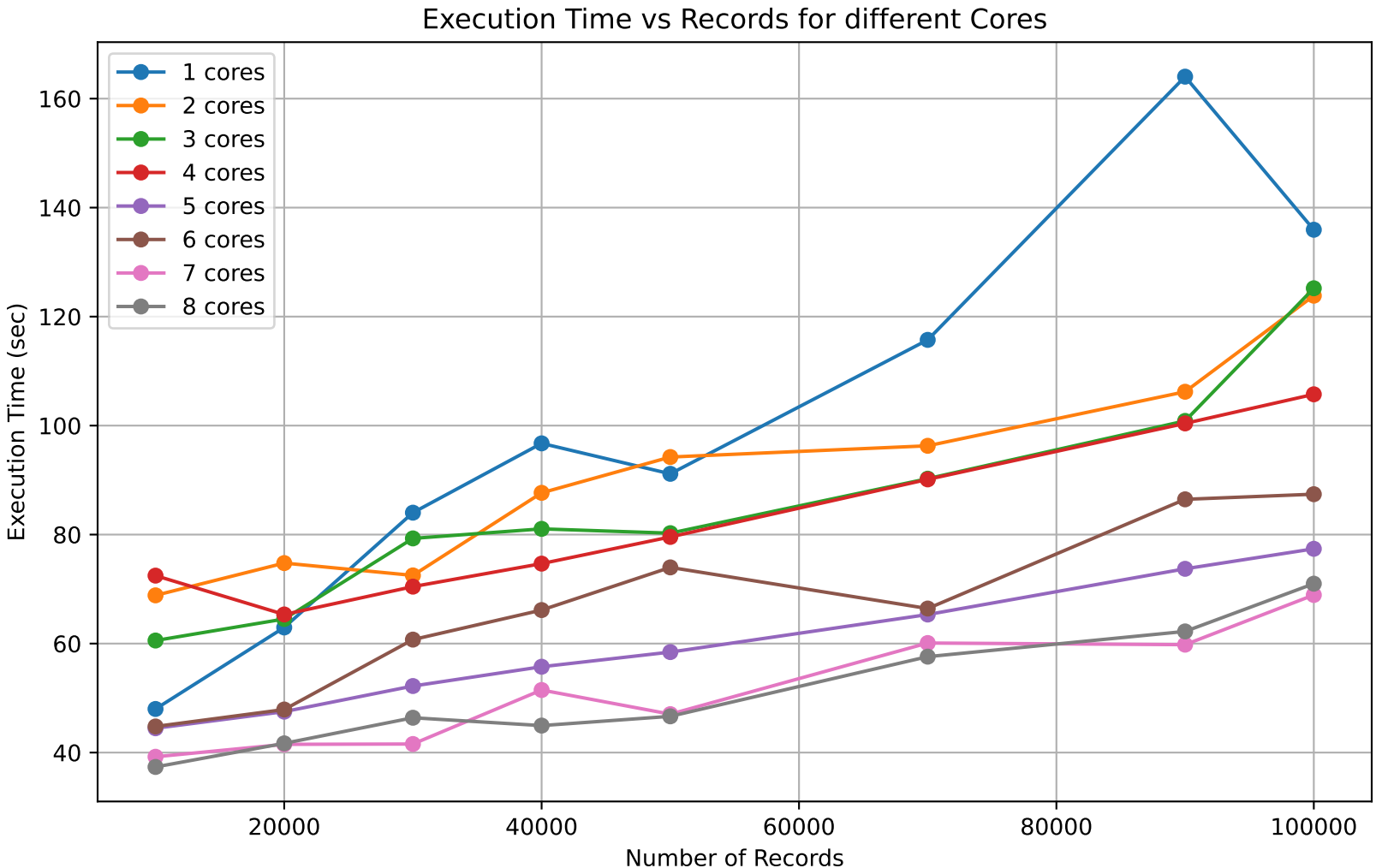
CS3.304 - Advanced Operating Systems
Assignment 4

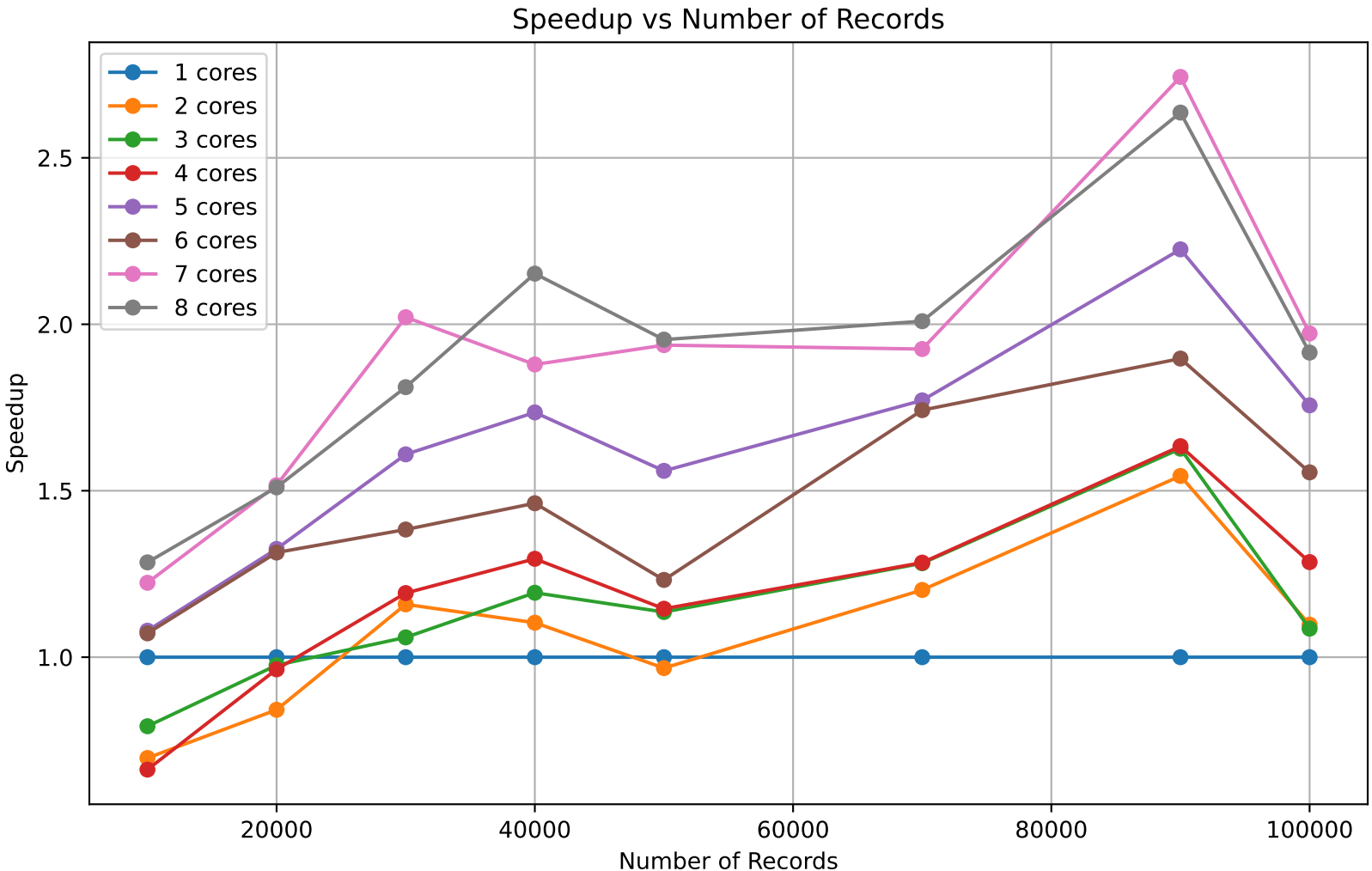
Deadlock Detection using MapReduce (PySpark)
Performance Analysis Report

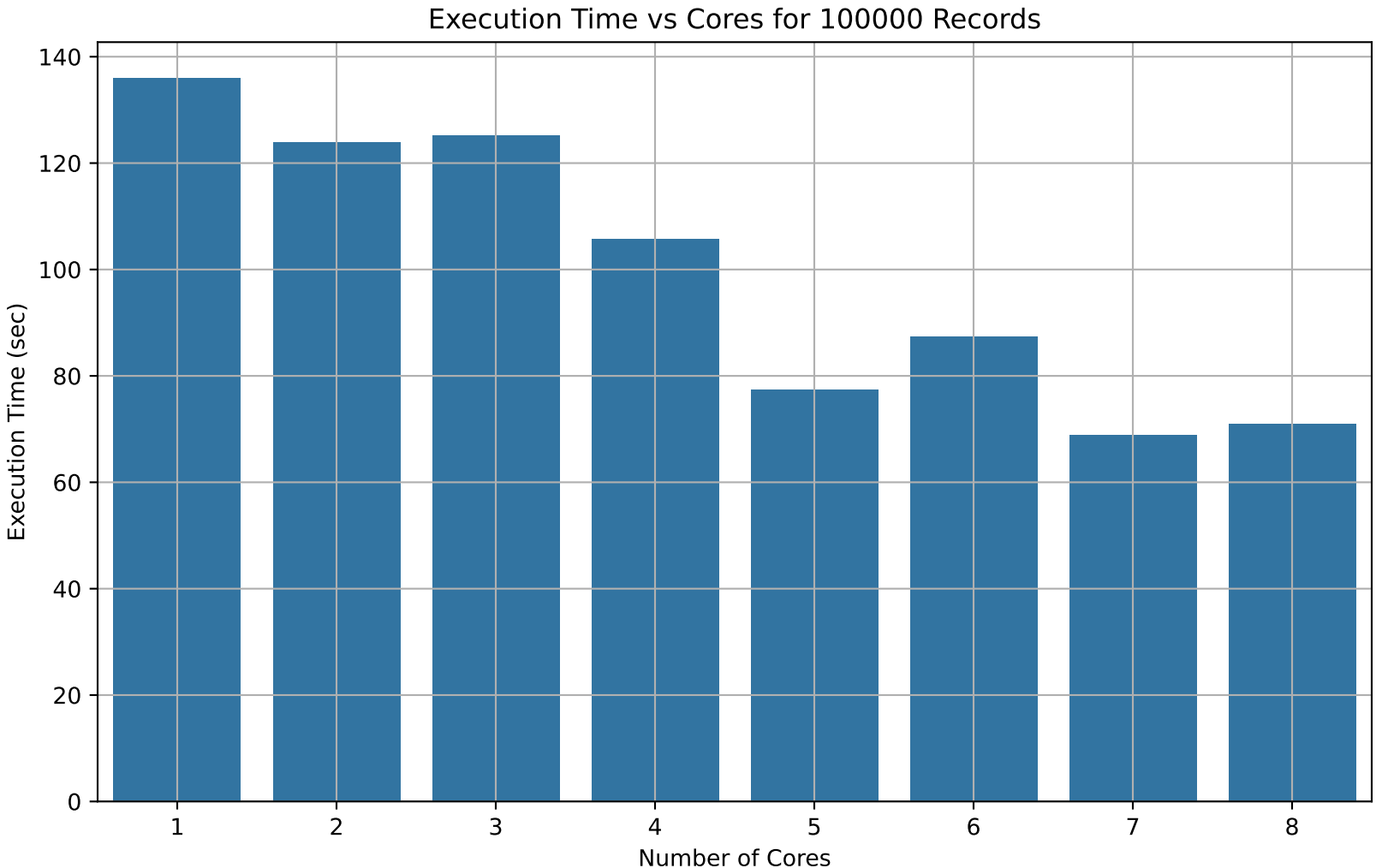
Submitted by: Shivam Patel
Roll No: 2035202030

Execution Time Table (Records vs Cores) Rows 1-8

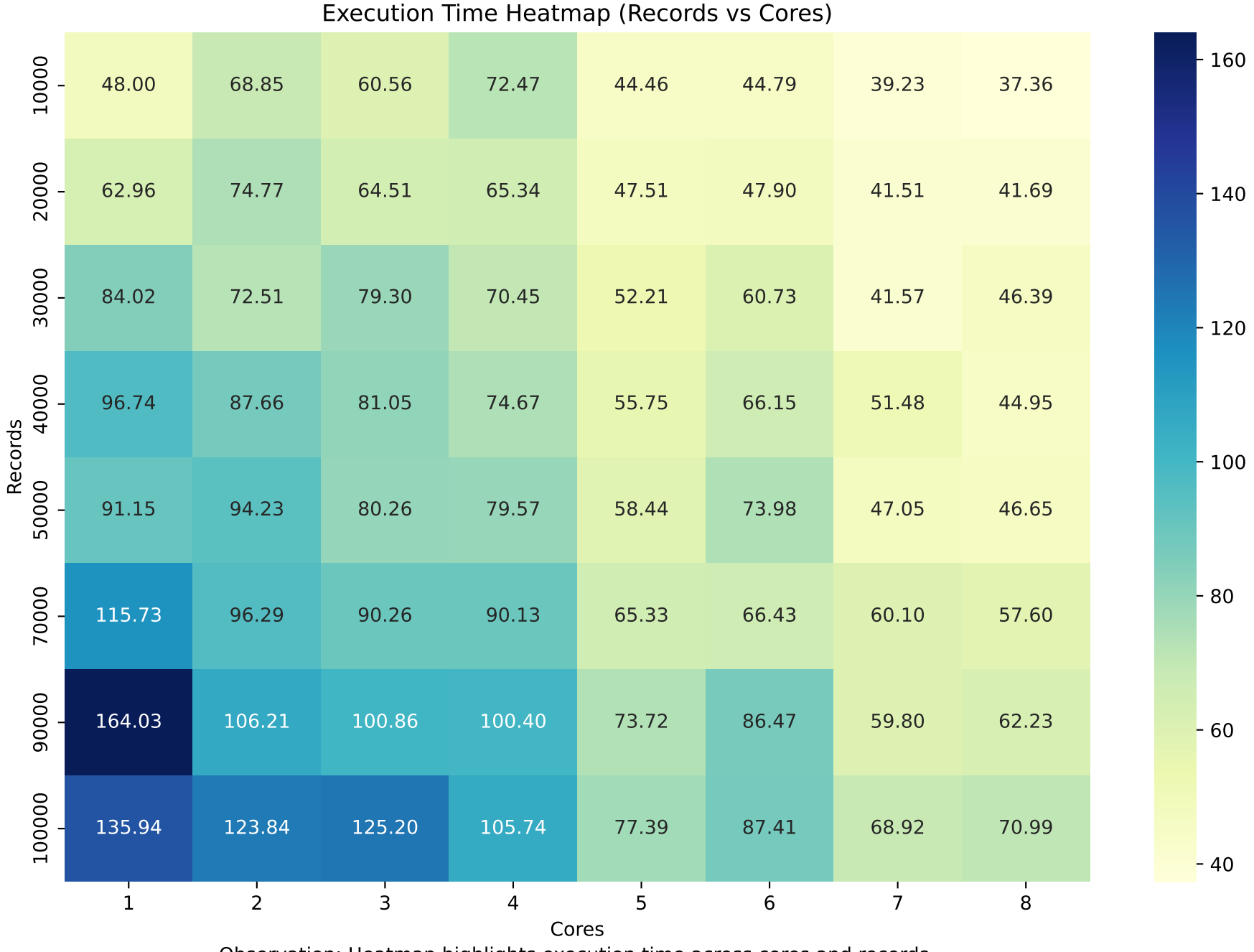
records	1	2	3	4	5	6	7	8
10000.0	48.0	68.85	60.56	72.47	44.46	44.79	39.23	37.36
20000.0	62.96	74.77	64.51	65.34	47.51	47.9	41.51	41.69
30000.0	84.02	72.51	79.3	70.45	52.21	60.73	41.57	46.39
40000.0	96.74	87.66	81.05	74.67	55.75	66.15	51.48	44.95
50000.0	91.15	94.23	80.26	79.57	58.44	73.98	47.05	46.65
70000.0	115.73	96.29	90.26	90.13	65.33	66.43	60.1	57.6
90000.0	164.03	106.21	100.86	100.4	73.72	86.47	59.8	62.23
100000.0	135.94	123.84	125.2	105.74	77.39	87.41	68.92	70.99


Execution Time vs Records for different Cores
1 cores
2 cores
3 cores
4 cores
5 cores
6 cores
7 cores
8 cores
Execution Time (sec)
160
140
120
100
80
60
40
20000
40000
60000
80000
100000
Number of Records


Speedup vs Number of Records
1 cores
2 cores
3 cores
4 cores
5 cores
6 cores
7 cores
8 cores
Speedup
2.5
2.0
1.5
1.0
20000
40000
60000
80000
100000
Number of Records


Execution Time vs Cores for 100000 Records
Execution Time (sec)
0
20
40
60
80
100
120
140
1
2
3
4
5
6
7
8
Number of Cores


Execution Time vs Cores for 10000 Records (Small Dataset)
Execution Time (sec)
0
10
20
30
40
50
60
70
1
2
3
4
5
6
7
8
Number of Cores



Observation: Heatmap highlights execution time across cores and records.
Optimal core utilization occurs when cores match dataset size for efficient parallelism.

Analysis & Observations

- Execution Time vs Records:
  - Execution time increases with the number of records.
  - For small datasets (10k–30k), time may fluctuate due to parallelization overheads.

- Speedup Analysis:
  - Adding cores initially improves performance.
  - Speedup is sub-linear; beyond 5–6 cores, gains diminish due to overhead and communication costs.

- Core Efficiency (Bar Chart):
  - Optimal core usage varies with dataset size.
  - For large datasets (100k records), 5–6 cores achieve the best trade-off between speedup and resource utilization.

- Heatmap Insights:
  - Highlights execution time across all cores and records.
  - Lower execution time is achieved when cores are balanced with workload.

- Execution Time Behavior:
  - Execution time does not always increase linearly for each core.
  - 1 core: 50k records (91.15s) < 40k records (96.74s)
  - 6 cores: 70k records (66.43s) < 40k (66.15s) or 50k (73.98s)
  - Fluctuations caused by task scheduling and I/O overhead in PySpark.

- Impact of Increasing Cores:
  - Small datasets (10k–30k) may see higher execution time with more cores.
  - Large datasets (70k–100k) benefit from parallelism, reducing time significantly.

- Speedup Analysis:
  - Sub-linear for 2–3 cores, near-linear at 5–6 cores.
  - Diminishing returns beyond 6–7 cores due to overhead.

- Resource Saturation:
  - Heatmap shows hotspots for mid-range cores and mid-sized datasets (40k–70k).
  - Efficiency can be computed as (speedup / cores) * 100%.

- Fluctuations in Execution Time:
  - Caused by Spark task scheduling, memory shuffling, and background processes.

- Technical Insights:
  - Parallel RDD processing depends on number of partitions, task size, and stage dependencies.
  - MapReduce cycle detection is iterative (max length 25).
  - Overhead factors include serialization, network transfer, and stage synchronization.
  - Scalability: beyond 5–6 cores for 100k records, gains are minimal.

Conclusion

- PySpark parallelization significantly improves performance for large datasets.
- Overheads prevent linear speedup, especially for small datasets.
- Careful tuning of cores versus dataset size is crucial for efficient distributed execution.
- Optimal cores: 5–6 for large datasets (100k records).
- Small datasets benefit from fewer cores to avoid parallelization overheads.